End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning – Assignment 2

1. Objective and architecture

This project implements a modular recommendation pipeline for a movie platform. It covers CSV ingestion, automated data-quality validation, preparation and chronological train/test splitting, reusable feature generation, two recommendation models, offline evaluation, experiment tracking and dependency-aware orchestration. Synthetic data is generated with a fixed seed so that the workflow is reproducible.

The architecture is: raw CSV sources → ingestion → validation → valid data / error quarantine → preparation and time split → feature tables and interaction matrix → popularity and item-item collaborative filtering → evaluation → experiment records and reports.

2. Data quality and preparation

The source tables contain user profiles, movie metadata and timestamped 1–5 ratings. Validation checks required columns, missing values, duplicate rows, rating type and range, timestamp parsing, duplicate interactions, and user/item referential integrity. Invalid interactions are written to a separate error CSV with reason codes; a JSON validation report records counts and status. Valid interactions are sorted chronologically. For users with at least two interactions, the latest interaction is held out for testing and earlier interactions form the training set.

3. Feature management

The reusable feature layer contains user activity count, average rating, preferred category and days since last interaction; item popularity, average rating and rating count; and interaction-level rating deviation from the user's average. A user-item rating matrix is also generated for collaborative filtering. Definitions, entity grain, availability and data types are documented in reports/feature_catalog.csv.

4. Models and evaluation

The baseline ranks items by distinct-user popularity, with mean rating as a tie-breaker. The personalized model uses item-item collaborative filtering: item vectors are compared using cosine similarity, and a user's observed ratings are used to score unseen items. Relevance is a held-out rating of at least 4. Ranking metrics use K=5: Precision@K, Recall@K, NDCG@K, catalog coverage and category-based intra-list diversity. MAE and RMSE are reported for the baseline's global-mean rating estimate; the item-item ranker does not predict explicit ratings.

model	k	precision_at_k	recall_at_k	ndcg_at_k	coverage	diversity	mae
Popularity baseline	5	0.0176470588235294	0.088235294117647	0.046103804861998	0.1375	0.7970588235294116	0.738071895424
Item-item collaborative filtering	5	0.0117647058823529	0.058823529411764 7	0.0406155809873958	0.65	0.7352941176470588	N/A

Observed results are generated by the included deterministic synthetic dataset and can be reproduced by running the pipeline. They should be interpreted as a demonstration of the evaluation workflow, not as a claim about real-world recommendation quality.

5. Orchestration, tracking and limitations

run_pipeline.py invokes the orchestrator, which executes stages in dependency order and writes a successful-run record. Parameters and metrics are always saved in reports/experiment_tracking.json; when MLflow is installed, the same run logs parameters, metrics and selected artifacts to MLflow. The workflow is intentionally implemented as a structured scheduled Python workflow rather than requiring an Airflow service.

Limitations include the small synthetic dataset, sparse user-item interactions, one-event-per-user holdout, and category-only diversity. A production system would require real behavioral data, time-window monitoring, incremental feature updates, privacy and access controls, and online A/B testing.

Execution

Install dependencies with `pip install -r requirements.txt`, then run `python run_pipeline.py`. The README documents input schemas, outputs and MLflow UI setup.